

Database Guide

Shared Postgres with pgvector. One database for the whole team, so data one person ingests is immediately visible to everyone.

Confirmed running: **PostgreSQL 18.6**, **pgvector 0.8.6**.

Connecting

Put the connection string in `.env` at the repo root:

```
DATABASE_URL=postgresql://<user>:<pass>@<host>.neon.tech/<db>?sslmode=require
```

Get the real value from the team channel. **It is never in git**. Keep `?sslmode=require` - the connection is rejected without it.

Check it works:

```
.venv/Scripts/python.exe backend/scripts/check_db.py --write  
.venv/Scripts/python.exe backend/scripts/check_db.py --read
```

Rules

- **Sync SQLAlchemy 2.0 only**. No `AsyncSession`, no `asyncpg`. See the async policy in `CLAUDE.md`.
- **No Alembic**. Schema changes happen by editing `models.py` and re-running `scripts/reset_db.py`. That drops and recreates. Tell the team before you run it.
- **No vector index**. The corpus is small enough that brute-force cosine is instant. An `ivfflat` index is a failure mode, not an optimisation.
- Only the backend owner edits `models.py`. Everyone else reads it.

The models

All of it lives in `backend/app/models.py`. One file, decided early, then frozen.

```
from datetime import datetime  
from sqlalchemy import (  
    String, Text, Integer, Boolean, ForeignKey, DateTime, JSON, Table, Column, func  
)  
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship  
from pgvector.sqlalchemy import Vector  
  
class Base(DeclarativeBase):  
    pass
```

Institution structure

```
class Department(Base):  
    __tablename__ = "departments"  
    id: Mapped[int] = mapped_column(primary_key=True)  
    name: Mapped[str] = mapped_column(String(200))  
  
course_prerequisites = Table(  
    "course_prerequisites",  
    Base.metadata,  
    Column("course_id", Integer, ForeignKey("departments.id", ondelete="CASCADE"), primary_key=True),  
    Column("prerequisite_id", Integer, ForeignKey("departments.id", ondelete="CASCADE"), primary_key=True)
```

```

        "course_prerequisites", Base.metadata,
        Column("course_id", ForeignKey("courses.id"), primary_key=True),
        Column("prerequisite_id", ForeignKey("courses.id"), primary_key=True),
    )

class Course(Base):
    __tablename__ = "courses"
    id: Mapped[int] = mapped_column(primary_key=True)
    code: Mapped[str] = mapped_column(String(20))
    title: Mapped[str] = mapped_column(String(200))
    department_id: Mapped[int | None] = mapped_column(ForeignKey("departments.id"))

```

Users and sessions

```

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    password_hash: Mapped[str] = mapped_column(String(255))
    full_name: Mapped[str] = mapped_column(String(200))
    role: Mapped[str] = mapped_column(String(20)) # student|teacher|admin
    course_id: Mapped[int | None] = mapped_column(ForeignKey("courses.id"))
    preferred_language: Mapped[str] = mapped_column(String(8), default="en")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                                    server_default=func.now())

class Session(Base):
    __tablename__ = "sessions"
    token: Mapped[str] = mapped_column(String(64), primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                                    server_default=func.now())

```

Password hashing uses `pbkdf2_sha256` from `hashlib` - pure Python, no native build. Not `bcrypt`: it fails to compile on Windows often enough to cost you an hour.

The token is `uuid4().hex`. Not a JWT. Logout deletes the row.

Curriculum

```

class Material(Base):
    __tablename__ = "materials"
    id: Mapped[int] = mapped_column(primary_key=True)
    course_id: Mapped[int] = mapped_column(ForeignKey("courses.id"))
    title: Mapped[str] = mapped_column(String(300))
    kind: Mapped[str] = mapped_column(String(20)) # syllabus|textbook|notes|assignment
    version: Mapped[int] = mapped_column(default=1)
    status: Mapped[str] = mapped_column(String(20), default="active") # active|archived
    page_count: Mapped[int] = mapped_column(default=0)
    uploaded_by_id: Mapped[int | None] = mapped_column(ForeignKey("users.id"))
    uploaded_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                                    server_default=func.now())
    ingest_status: Mapped[str] = mapped_column(String(20), default="queued")
    chunk_count: Mapped[int] = mapped_column(default=0)

class Chunk(Base):
    __tablename__ = "chunks"
    id: Mapped[int] = mapped_column(primary_key=True)
    material_id: Mapped[int] = mapped_column(ForeignKey("materials.id"), index=True)
    page_no: Mapped[int] = mapped_column(index=True)
    chapter: Mapped[str | None] = mapped_column(String(300))

```

```

char_start: Mapped[int] = mapped_column(default=0)
char_end: Mapped[int] = mapped_column(default=0)
text: Mapped[str] = mapped_column(Text)
embedding: Mapped[list[float]] = mapped_column(Vector(384))

```

page_no is not optional. Every citation the student sees resolves through it, and "Show Source" is quoted directly in the problem statement.

kind="assignment" is what powers the graded-work guardrail. Assignment material is searchable for **matching**, but never quoted as an answer.

Topics and concepts

```

class Topic(Base):
    __tablename__ = "topics"
    id: Mapped[int] = mapped_column(primary_key=True)
    course_id: Mapped[int] = mapped_column(ForeignKey("courses.id"))
    name: Mapped[str] = mapped_column(String(200))

class Concept(Base):
    __tablename__ = "concepts"
    id: Mapped[int] = mapped_column(primary_key=True)
    topic_id: Mapped[int] = mapped_column(ForeignKey("topics.id"))
    name: Mapped[str] = mapped_column(String(200))
    prerequisite_course_id: Mapped[int | None] = mapped_column(ForeignKey("courses.id"))

```

Diagnostic, gaps, mastery

```

class DiagnosticItem(Base):
    __tablename__ = "diagnostic_items"
    id: Mapped[int] = mapped_column(primary_key=True)
    course_id: Mapped[int] = mapped_column(ForeignKey("courses.id"))
    concept_id: Mapped[int] = mapped_column(ForeignKey("concepts.id"))
    prompt: Mapped[str] = mapped_column(Text)
    kind: Mapped[str] = mapped_column(String(20), default="mcq")
    options: Mapped[dict | None] = mapped_column(JSON)
    correct_answer: Mapped[str] = mapped_column(String(300))

class Gap(Base):
    __tablename__ = "gaps"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    concept_id: Mapped[int] = mapped_column(ForeignKey("concepts.id"))
    detected_from: Mapped[str] = mapped_column(String(30)) # diagnostic|syllabus_upload|practice
    status: Mapped[str] = mapped_column(String(20), default="open")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                                    server_default=func.now())

class Mastery(Base):
    __tablename__ = "mastery"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    concept_id: Mapped[int] = mapped_column(ForeignKey("concepts.id"))
    state: Mapped[str] = mapped_column(String(20), default="untested") # solid|shaky|untested
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                                    server_default=func.now())

```

There is deliberately **no score column** and **no time-on-task column** anywhere. The problem statement asks for a gap list, not a grade, and for dashboards focused on misconceptions rather than surveillance. Do not add them.

Practice and attempts

```
class PracticeSet(Base):
    __tablename__ = "practice_sets"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    gap_id: Mapped[int] = mapped_column(ForeignKey("gaps.id"))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                                    server_default=func.now())

class PracticeItem(Base):
    __tablename__ = "practice_items"
    id: Mapped[int] = mapped_column(primary_key=True)
    practice_set_id: Mapped[int] = mapped_column(ForeignKey("practice_sets.id"))
    gap_id: Mapped[int] = mapped_column(ForeignKey("gaps.id"))
    problem_type: Mapped[str] = mapped_column(String(60), index=True)
    prompt: Mapped[str] = mapped_column(Text)
    kind: Mapped[str] = mapped_column(String(20), default="mcq")
    options: Mapped[dict | None] = mapped_column(JSON)
    correct_answer: Mapped[str] = mapped_column(String(300))

class Attempt(Base):
    __tablename__ = "attempts"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    practice_item_id: Mapped[int] = mapped_column(ForeignKey("practice_items.id"))
    answer: Mapped[str] = mapped_column(String(300))
    correct: Mapped[bool] = mapped_column(Boolean)
    at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
```

`correct_answer` never leaves the server. Strip it in the response model.

Misconceptions - the heart of the demo

```
class Misconception(Base):
    __tablename__ = "misconceptions"
    id: Mapped[int] = mapped_column(primary_key=True)
    topic_id: Mapped[int] = mapped_column(ForeignKey("topics.id"))
    problem_type: Mapped[str] = mapped_column(String(60), index=True)
    label: Mapped[str] = mapped_column(String(300))
    description: Mapped[str] = mapped_column(Text)
    wrong_answer_pattern: Mapped[str | None] = mapped_column(String(300))

class MisconceptionDiagnosis(Base):
    __tablename__ = "misconception_diagnoses"
    id: Mapped[int] = mapped_column(primary_key=True)
    attempt_id: Mapped[int] = mapped_column(ForeignKey("attempts.id"))
    misconception_id: Mapped[int] = mapped_column(ForeignKey("misconceptions.id"), index=True)
    confirmed: Mapped[bool | None] = mapped_column(Boolean, default=None)
    at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
```

`confirmed` is deliberately three-state: `None` means asked but not answered, `True` confirmed, `False` denied. **Only `True` feeds the teacher heatmap**. Denied diagnoses are stored but excluded from every teacher aggregate - that is what makes the number honest.

Teacher-facing tables

```
class UncertaintyFlag(Base):
    __tablename__ = "uncertainty_flags"
    id: Mapped[int] = mapped_column(primary_key=True)
```

```

question: Mapped[str] = mapped_column(Text)
alignment_score: Mapped[float] = mapped_column()
reason: Mapped[str] = mapped_column(String(100))
topic_id: Mapped[int | None] = mapped_column(ForeignKey("topics.id"))
status: Mapped[str] = mapped_column(String(20), default="open")
occurred_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                              server_default=func.now())

class ReteachUnit(Base):
    __tablename__ = "reteach_units"
    id: Mapped[int] = mapped_column(primary_key=True)
    misconception_id: Mapped[int] = mapped_column(ForeignKey("misconceptions.id"))
    title: Mapped[str] = mapped_column(String(300))
    body: Mapped[str] = mapped_column(Text)
    status: Mapped[str] = mapped_column(String(20), default="draft") # draft|assigned
    approved_by_id: Mapped[int | None] = mapped_column(ForeignKey("users.id"))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                              server_default=func.now())

class SourcedContent(Base):
    __tablename__ = "sourced_content"
    id: Mapped[int] = mapped_column(primary_key=True)
    source_url: Mapped[str] = mapped_column(String(500))
    title: Mapped[str] = mapped_column(String(300))
    excerpt: Mapped[str] = mapped_column(Text)
    found_for_gap: Mapped[str] = mapped_column(String(200))
    status: Mapped[str] = mapped_column(String(20), default="pending")
    found_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                              server_default=func.now())

class AuditLog(Base):
    __tablename__ = "audit_log"
    id: Mapped[int] = mapped_column(primary_key=True)
    actor_id: Mapped[int | None] = mapped_column(ForeignKey("users.id"))
    action: Mapped[str] = mapped_column(String(60))
    target: Mapped[str] = mapped_column(String(120))
    detail: Mapped[dict | None] = mapped_column(JSON)
    at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

```

UncertaintyFlag has no `user_id`. Teacher views must be anonymous, and the cheapest way to guarantee that is to never store the link.

A **draft** reteach unit must never appear in any student query. The approval gate is the human-in-the-loop story.

Creating and resetting

```

# backend/scripts/reset_db.py
from sqlalchemy import text
from app.db import get_engine
from app.models import Base

engine = get_engine()
with engine.begin() as conn:
    conn.execute(text("CREATE EXTENSION IF NOT EXISTS vector"))
Base.metadata.drop_all(engine)
Base.metadata.create_all(engine)
print("schema recreated")

```

This is destructive and the database is shared. **Announce it before running it.**

Querying

Keep it to `.all()`, `.filter()`, `.order_by()`. No joins beyond what you need, no eager-loading tricks.

```
from sqlalchemy import select

rows = db.execute(
    select(Gap).where(Gap.user_id == user.id, Gap.status == "open")
).scalars().all()
```

Vector search - nearest chunks to a query embedding:

```
rows = db.execute(
    select(CHUNK, CHUNK.embedding.cosine_distance(query_vec).label("dist"))
    .order_by("dist")
    .limit(5)
).all()
```

`cosine_distance` returns distance, so **smaller is better**. Similarity is `1 - distance`. Getting this backwards silently inverts your alignment score, so check the numbers on a query you know the answer to.

Seeding

`backend/scripts/seed.py` is a first-class deliverable, not an afterthought. It must be **idempotent** - safe to run twice - and it must create:

- one department, one course, one prerequisite course
- three users: student, teacher, admin
- topics and concepts
- diagnostic items that produce a predictable gap
- practice items with known `problem_type`
- misconceptions with `wrong_answer_pattern` matching those items
- some pre-existing confirmed diagnoses so the teacher heatmap is not empty on demo day
- a few pending `SourcedContent` rows and one `ReteachUnit`

An empty teacher dashboard on demo day looks broken even when it is correct.